

EXAM POV NOTES (Q&A FORMAT)

1. Knowledge-Based Agents

Question 1 (10 marks): Explain the declarative approach to building knowledge-based agents. Describe the roles of the knowledge base (KB) and inference engine, and illustrate with the Tell-Ask paradigm how agents can derive actions from percepts. Include an example pseudocode for a simple KB agent.

The declarative approach separates domain-specific content from general algorithms, enabling modular and reusable agent design in logical agents.

- **Declarative Programming Principle:** Facts and rules are stated explicitly without procedural code, allowing separation of "what" from "how" in agent behavior. This principle ensures that the agent's knowledge is expressed in a formal language, making it easy to modify or extend without rewriting core logic.
- **Knowledge Base Role:** KB stores sentences representing world assertions in a formal language, serving as the factual repository for reasoning. It acts as the central memory, where percepts are added as new sentences, maintaining a consistent model of the environment over time.
- **Inference Engine Role:** Applies domain-independent algorithms to derive new knowledge or actions from the KB, ensuring logical consistency. This engine performs deductions like entailment checks, allowing the agent to infer hidden facts without domain-specific tweaks.
- **Tell Operation:** Adds new percept-derived sentences to the KB, updating the agent's world model incrementally. For instance, a percept like "breeze detected" becomes a sentence $\neg \text{Breeze_}\{1,1\}$ or similar, integrating real-time sensory data seamlessly.
- **Ask Operation:** Queries the KB for entailed sentences, such as possible actions, to decide the next move. It uses inference to check if an action like "forward" follows from current knowledge, providing a reasoned response.
- **Modularity Benefit:** Changes in domain knowledge only require KB updates, without altering the inference engine. This allows quick adaptations, such as switching from Wumpus World rules to medical diagnostics, enhancing flexibility.
- **Reusability Across Domains:** Same engine works for different KBs, like Wumpus World versus medical diagnosis. It promotes code reuse, reducing development time and errors in diverse AI applications.
- **Pseudocode Structure:** Function $\text{KB-AGENT}(\text{percept})$ initializes static KB and time counter for sequential processing. This structure handles time-dependent states, ensuring actions build on prior percepts chronologically.
- **Percept Integration:** $\text{TELL}(\text{KB}, \text{MAKE-PERCEPT-SENTENCE}(\text{percept}, t))$ incorporates sensory data as logical sentences. This step translates raw inputs like smells into formal assertions, grounding the agent's perception.
- **Action Derivation:** $\text{ASK}(\text{KB}, \text{MAKE-ACTION-QUERY}(t))$ retrieves entailed actions, followed by TELL for action effects; increment time t . This closes the loop, updating KB with action outcomes for future inferences.

Question 2 (5 marks): Compare viewing agents at the knowledge level versus the implementation level. Why is the knowledge level abstraction useful for agent design?

The knowledge level abstracts agent behavior to "what is known," while implementation focuses on "how it's computed," aiding high-level design.

- **Knowledge Level Focus:** Emphasizes the agent's beliefs and goals as sentences, independent of internal data structures. This view treats the agent as a rational entity based on what it knows, ignoring computational details for conceptual clarity.

- **Implementation Level Focus:** Deals with algorithms and KB representations, like arrays or graphs for manipulation. It concerns efficiency, such as how resolution is coded, but risks obscuring overall rationality.
- **Abstraction Utility:** Allows reasoning about rationality without low-level details, simplifying verification. Designers can validate if knowledge leads to correct actions before optimizing code.
- **Design Flexibility:** Changes in implementation don't affect knowledge-level semantics, enabling optimization. This separation supports iterative improvements without re-evaluating core logic.
- **Scalability Advantage:** Facilitates modular testing of knowledge content separately from code efficiency. Large systems benefit as knowledge grows independently of performance tweaks.

Question 3 (2 marks): Define a knowledge base and a sentence in the context of knowledge-based agents.

Core components of logical agents, where sentences encode world facts for inference.

- **Knowledge Base Definition:** A set of sentences in a formal language, representing the agent's accumulated assertions about the world. It evolves through tells and asks, forming the basis for all deductions in dynamic environments.
- **Sentence Definition:** An expression in the knowledge representation language that asserts a fact or relation, like "There is a pit at [2,2]." Sentences must be well-formed to enable valid inference, ensuring precise world modeling.

2. Wumpus World

Question 4 (10 marks): Provide a detailed PEAS description for the Wumpus World, including performance measures, environment characteristics, actuators, and sensors. Explain how percepts like breeze and glitter inform the agent about hidden aspects.

PEAS framework defines the task environment for the Wumpus agent, using percepts to infer unseen dangers and rewards.

- **Performance Measure - Gold Reward:** +1000 points for grabbing gold, incentivizing treasure-seeking behavior. This high reward drives the agent to prioritize gold location over mere survival.
- **Performance Measure - Death Penalty:** -1000 points for falling into a pit or being eaten by Wumpus, penalizing risky moves. It enforces caution, making avoidance strategies central to optimal play.
- **Performance Measure - Step Cost:** -1 per action step, encouraging efficient paths to minimize time. This simulates real resource limits, pushing for shortest safe routes.
- **Performance Measure - Arrow Cost:** -10 for shooting, discouraging unnecessary Wumpus hunts. It balances aggression with conservation, as the single arrow is irreplaceable.
- **Environment - Adjacency Rules:** Squares next to pits are breezy; next to Wumpus are smelly, providing indirect danger cues. These rules create a logic puzzle, where local percepts reveal global layout.
- **Environment - Glitter Indicator:** Glitter appears only if gold is in the current square, signaling immediate reward. It serves as a direct, unambiguous cue for grabbing actions.
- **Actuators - Movement:** Forward advances one square; left/right turns 90 degrees for navigation. These basic moves allow grid traversal while respecting bump feedback.

- **Actuators - Interaction:** Grab picks up gold; Release drops it; Shoot fires arrow if facing Wumpus. Interaction actuators enable goal achievement and defense.
- **Sensors - Percept Types:** Breeze detects nearby pits; Glitter detects gold; Smell detects nearby Wumpus. Bump signals walls; Scream confirms Wumpus kill.
- **Inference from Percepts:** Breeze in [2,1] implies pit in adjacent [1,1], [2,2], [3,1], or [2,0], enabling safe path deduction. Percepts like these allow probabilistic or logical mapping of hazards.

Question 5 (5 marks): Characterize the Wumpus World environment across the six dimensions (observable, deterministic, episodic, static, discrete, single-agent). Justify each classification with reference to the problem setup.

Wumpus World is a partially observable, deterministic grid world, classified by PEAS properties for agent design.

- **Observable: Partially Observable:** Agent perceives only current square via local sensors like breeze, not the full grid. This limitation forces inference-based exploration rather than direct visibility.
- **Deterministic: Fully Deterministic:** Actions like forward always move one square if no bump, with fixed outcomes. No randomness means predictable results from the same state-action pair.
- **Episodic: Sequential:** Actions form sequences affecting future states, like arrow use preventing later shots. Past decisions influence ongoing episodes, unlike independent tasks.
- **Static: Fully Static:** Wumpus and pits remain fixed; only agent moves during exploration. Environment doesn't change without agent input, simplifying planning.
- **Discrete: Discrete:** Fixed 4x4 grid with distinct squares and finite action/percept sets. All elements are countable, avoiding continuous state spaces.

Question 6 (5 marks): Describe the exploration process in Wumpus World, highlighting how the agent uses percepts to infer safe and dangerous squares. Use a small grid example to illustrate.

Agent explores via logical inference from percepts, marking safe/dangerous cells to avoid death.

- **Initial Percept Check:** At [1,1] with no percepts, infer [1,1] safe; move right to [2,1]. This establishes a known safe base for further deductions.
- **Breeze Inference:** Breeze at [2,1] implies pit in adjacent [1,1], [2,2], [3,1], or [2,0]; [1,1] safe by prior. It narrows possibilities, marking [2,2] risky until disproven.
- **Safe Square Marking:** No breeze/smell at start marks [1,1] safe; propagate to non-adjacent cells. Logical rules like $\neg \text{Breeze} \Rightarrow \neg \text{Pit}$ allow chaining safety proofs.
- **Risk Assessment:** Avoid [2,2] if breeze suggests possible pit; climb out if all safe inferred. This decision tree balances exploration with survival probabilities.
- **Grid Example Illustration:** In 2x2 grid, breeze at [1,2] $\rightarrow$ pit likely at [2,2]; safe path: [1,1] $\rightarrow$ [2,1]. Agent infers [2,2] dangerous, opting for climb-out.

Question 7 (2 marks): What is the performance penalty for using the arrow in Wumpus World?

Resource scarcity mechanic to promote cautious shooting.

- **Penalty Value:** -10 points deducted for firing the single arrow, regardless of hit. This cost reflects the arrow's uniqueness, making shots a high-stakes choice.
- **Rationale:** Encourages inference-based Wumpus location before shooting. It prevents spam firing, forcing precise orientation and timing.

3. Inference Concepts

Question 8 (10 marks): Define soundness and completeness in inference procedures. Explain the ground connection in logical reasoning and its importance for agents in real-world environments. Discuss implications for knowledge-based systems.

Soundness ensures truth preservation; completeness guarantees full entailment coverage; grounding links logic to reality.

- **Soundness Definition:** Inference procedure i is sound if $KB \vdash_i \alpha$ implies $KB \models \alpha$, deriving only true sentences. It prevents false conclusions, maintaining reliability in agent decisions.
- **Completeness Definition:** Procedure i is complete if $KB \models \alpha$ implies $KB \vdash_i \alpha$, deriving all entailed sentences. It ensures no missed truths, capturing full logical potential.
- **Truth Preservation:** Soundness guarantees derived α holds in all models where KB is true. This property is crucial for trustworthy AI outputs in uncertain worlds.
- **Exhaustive Derivation:** Completeness ensures no missed logical consequences from KB . It allows exhaustive exploration of implications for optimal actions.
- **Grounding Concept:** Links logical symbols to real-world referents, like mapping "Pit(x)" to actual holes. Without it, abstract logic remains disconnected from percepts.
- **Semantic Grounding:** Assigns interpretations to constants/predicates for real applicability. This mapping enables sensors to feed into KB meaningfully.
- **Agent Importance:** Enables percept-to-action mapping in dynamic environments like Wumpus World. Grounded inferences turn abstract rules into navigable plans.
- **KB System Implications:** Sound+complete procedures build reliable agents, but trade-offs exist in efficiency. Real systems often prioritize soundness for safety.
- **Real-World Challenges:** Incomplete grounding leads to faulty inferences, e.g., misinterpreting breeze. Agents must refine mappings through learning.
- **Design Trade-off:** Prefer sound procedures; completeness may be computationally expensive. In practice, hybrid approaches balance both for scalability.

Question 9 (5 marks): Differentiate between entailment ($KB \models \alpha$) and derivation ($KB \vdash_i \alpha$). How do soundness and completeness relate to these?

Entailment is semantic necessity; derivation is syntactic proof; properties bridge them.

- **Entailment ($KB \models \alpha$):** α true in every model where KB is true; model-based necessity. It's the gold standard for logical consequence, independent of procedure.
- **Derivation ($KB \vdash_i \alpha$):** α follows from KB via procedure i steps; proof-based. It relies on syntactic rules, potentially varying by algorithm.
- **Soundness Relation:** Ensures derivation implies entailment, avoiding false positives. A sound i never oversteps beyond true implications.
- **Completeness Relation:** Ensures entailment implies derivation, capturing all truths. It guarantees the procedure finds every valid consequence.
- **Bridge Role:** Sound+complete i makes derivation equivalent to entailment. This equivalence validates automated reasoning tools.

Question 10 (2 marks): What is the Grounding connection in inference?

Bridges abstract logic to concrete reality for practical agent reasoning.

- **Definition:** Mapping logical expressions to real-world interpretations or referents. It translates symbols like "Breeze" to sensor readings for applicability.
- **Purpose:** Ensures inferences align with environmental truths, like percept symbols to sensors. Without grounding, logic fails in physical contexts.

4. Propositional Logic: Syntax and Semantics

Question 11 (10 marks): Outline the syntax of propositional logic, including atomic and complex sentences, connectives, literals, conjunctions, disjunctions, implications, and biconditionals. Provide examples from Wumpus World (e.g., rules for pits and breezes) and explain operator precedence.

Syntax defines well-formed sentences using symbols and connectives for logical expression in agents.

- **Atomic Sentences:** Propositions like $P_{\{1,2\}}$ (pit in $[1,2]$), true/false without structure. They represent basic facts, foundational for building complex knowledge.
- **Complex Sentences:** Built from atomics via connectives, e.g., $\neg P_{\{1,2\}}$ (no pit). This composition allows expressing relationships between facts.
- **Negation ($\neg$):** Unary; $\neg B_{\{1,1\}}$ means no breeze in $[1,1]$. It inverts truth, essential for absences like safe squares.
- **Conjunction ($\wedge$):** Binary; $B_{\{1,1\}} \wedge S_{\{2,1\}}$ (breeze and smell). Combines facts for joint conditions in rules.
- **Disjunction ($\vee$):** Binary; $P_{\{2,2\}} \vee P_{\{3,1\}}$ (pit in either square). Represents alternatives, like possible pit locations.
- **Implication ($\Rightarrow$):** Binary; $B_{\{2,1\}} \Rightarrow (P_{\{1,1\}} \vee P_{\{2,2\}} \vee P_{\{3,1\}})$ (breeze implies adjacent pit). Models causal links in environments.
- **Biconditional ($\Leftrightarrow$):** Binary; $B_{\{x,y\}} \Leftrightarrow \exists$ adjacent pit (exact equivalence). Ensures if-and-only-if relations for precise modeling.
- **Literals:** Atomic or negated atomic, e.g., $\neg P_{\{2,2\}}$ as positive literal form. They form clauses in normal forms for inference.
- **Precedence Order:** $\neg > \wedge > \vee > \Rightarrow > \Leftrightarrow$; parentheses override, e.g., $(A \Rightarrow B) \wedge C$. This hierarchy avoids ambiguity in sentence parsing.
- **Wumpus Example:** R1: $\neg B_{\{1,1\}}$ (no breeze at start); R2: $B_{\{2,1\}} \Leftrightarrow (P_{\{2,2\}} \vee P_{\{3,1\}} \vee P_{\{1,1\}})$. These rules encode percept-to-hazard mappings.

Question 12 (5 marks): Describe the semantics of propositional logic using models and truth tables. Evaluate the truth value of the sentence $\neg P_{\{1,2\}} \wedge (P_{\{2,2\}} \vee P_{\{3,1\}})$ under a model where $P_{\{1,2\}}=\text{false}$, $P_{\{2,2\}}=\text{false}$, $P_{\{3,1\}}=\text{true}$.

Semantics assigns truth via interpretations; truth tables enumerate all possibilities.

- **Models Definition:** Assignments of true/false to each proposition symbol, e.g., $\{P=\text{true}, Q=\text{false}\}$. Models define possible worlds for evaluation.
- **Satisfiability:** Sentence true in a model if evaluation yields true. Unsatisfiable if false in all models, like contradictions.
- **Validity:** True in all models, like tautologies. Valid sentences hold universally, independent of specifics.
- **Entailment via Models:** $KB \models \alpha$ if no model makes KB true and α false. This checks semantic implication exhaustively.
- **Evaluation Result:** $\neg \text{false}=\text{true}$; $(\text{false} \vee \text{true})=\text{true}$; $\text{true} \wedge \text{true}=\text{true}$. The sentence holds true under this model assignment.

Question 13 (2 marks): What is a literal in propositional logic?

Basic building block for clauses in inference.

- **Definition:** An atomic sentence or its negation, e.g., P or $\neg P$. Literals are the simplest non-composite expressions.
- **Role:** Forms clauses in CNF for resolution. They enable efficient matching in proof procedures.

5. Propositional Entailment and Inference

Question 14 (10 marks): Explain how entailment is determined in propositional logic using models. Illustrate with the Wumpus World example: Given KB with rules R1-R5 for pits and breezes, show why $KB \models \neg P_{\{2,2\}}$ but not $KB \models P_{\{2,2\}}$. Discuss the role of truth tables in inference.

Model enumeration checks necessity; Wumpus KB uses percept rules for pit negation.

- **Model-Based Entailment:** α entailed if true in all KB-satisfying models. This semantic test confirms necessity from knowledge.
- **Unsatisfiability Test:** $KB \wedge \neg \alpha$ unsatisfiable implies entailment. Refutation proves by contradiction absence of counter-models.
- **Enumeration Process:** List all 2^n models for n symbols. Systematic checking reveals consistent truths.
- **Wumpus KB Setup:** R1: $\neg B_{\{1,1\}}$; R2: $B_{\{1,1\}} \Leftrightarrow (P_{\{1,2\}} \vee P_{\{2,1\}})$; etc. These encode breeze-pit equivalences.
- **Percept Scenario:** No breeze [1,1], breeze [2,1] $\rightarrow$ symbols $B_{\{1,1\}}=false$, $B_{\{2,1\}}=true$. Fixes model constraints from observations.
- **$\neg P_{\{2,2\}}$ Entailment:** All models satisfying KB make $P_{\{2,2\}}=false$ to fit breezes. No counter-model exists for safety proof.
- **Non-Entailment $P_{\{2,2\}}$:** Models exist where KB true and $P_{\{2,2\}}=true$ (pit elsewhere). Multiple possibilities prevent certainty.
- **Truth Table Role:** Exhaustively checks models; for 4 symbols, 16 rows. Visualizes all combinations for small cases.
- **Efficiency Limit:** Exponential time $O(2^n)$, impractical for large n . Limits use to toy problems like early Wumpus grids.
- **Inference Aid:** Tables verify small-KB entailments like Wumpus safety. They teach logical structure intuitively.

Question 15 (5 marks): Describe the TT-ENTAILS? algorithm for propositional entailment. Why is it sound and complete, and what is its complexity ($O(2^n)$ for n symbols)?

Truth table enumeration algorithm for exact entailment checking.

- **Algorithm Steps:** Enumerate all models; check if any satisfies KB but not α . Scans for counterexamples methodically.
- **Return Value:** False if such model found; true otherwise (entailed). Clear yes/no for query resolution.
- **Soundness Reason:** Only returns true when no counter-model, preserving truth. Avoids invalid derivations inherently.
- **Completeness Reason:** Checks all models, capturing every entailment. No omissions in exhaustive search.
- **Complexity:** $O(2^n * \text{poly}(n))$ time, exponential in symbols. Scales poorly but exact for verification.

Question 16 (5 marks): Compare forward chaining (FC) and backward chaining (BC) for propositional inference. When is BC more efficient, and provide a simple example with Horn clauses.

Chaining algorithms for Horn KB; FC data-driven, BC goal-driven.

- **FC Overview:** Starts from facts, applies rules to derive new facts forward. Builds knowledge breadth-first from percepts.
- **BC Overview:** Starts from query, works backward to match known facts. Focuses depth-first on goal satisfaction.
- **Efficiency Comparison:** FC better for many queries; BC for few/single queries. FC computes once; BC prunes irrelevant paths.
- **BC Efficiency Case:** When query-specific, avoids irrelevant derivations. Ideal for diagnostic asks in large KBs.
- **Horn Example:** KB: $P \Rightarrow Q$, $Q \Rightarrow R$; Query R ? BC: $R \leftarrow Q \leftarrow P$ (true if P known). Chains backward efficiently for single goal.

Question 17 (2 marks): What is a model in propositional logic semantics?

Interpretation assigning truth values for evaluation.

- **Definition:** Complete assignment of T/F to all proposition symbols. Represents a possible world state.
- **Usage:** Determines sentence truth, e.g., $\{P=\text{true}\}$ models P . Essential for semantic checks.

6. Resolution in Propositional Logic

Question 18 (5 marks): Explain the resolution rule in propositional logic and its role in proving unsatisfiability. Describe the resolution closure $RC(S)$ and the Ground Resolution Theorem.

Contradiction-based refutation for complete inference.

- **Resolution Rule:** From $(\neg I \vee A)$ and $(I \vee B)$, derive $(A \vee B)$ where I unifies literals. Resolves opposites to simplify clauses.
- **Unsatisfiability Role:** Repeated resolution on $KB \wedge \neg\alpha$ yields empty clause if unsatisfiable. Proves entailment by contradiction.
- **$RC(S)$ Definition:** Set of all clauses from repeated resolutions on S . Closure exhausts all derivable disjunctions.
- **Ground Theorem:** $RC(KB \wedge \neg\alpha)$ empty iff $KB \models \alpha$, proving completeness. Validates refutation as full method.
- **Refutation Strategy:** Negate query, resolve to contradiction for entailment. Efficient for automated theorem proving.

Question 19 (2 marks): Why is resolution complete for propositional logic?

Guarantees derivation of all entailments via exhaustive closure.

- **Completeness Proof:** Ground Resolution Theorem shows unsatisfiability detection. Covers all propositional cases in CNF.
- **CNF Requirement:** Works on clausal form, covering all propositional cases. Transformability ensures universality.

7. Pros/Cons of Propositional Logic and Transition to FOL

Question 20 (5 marks): Discuss the advantages and disadvantages of propositional logic, using the Wumpus World as an example (e.g., rules for finding pits or Wumpus). Why does it lack expressive power compared to natural language?

Simple but limited logic; excels in decidability, fails in compactness.

- **Advantage - Declarative:** Sentences directly map to facts, e.g., $B_{\{x,y\}}$ for breeze. Easy to read and assert world states.
- **Advantage - Inference Methods:** Supports sound/complete procedures like resolution. Decidable with finite checks.
- **Disadvantage - Scalability:** Wumpus needs 16+ rules for breezes, exponential growth. Repetition bloats KB for grids.
- **Disadvantage - No Variables:** Separate propositions per square, unlike $\forall x$ in FOL. Lacks generalization for patterns.
- **Expressiveness Gap:** Can't generalize "adjacent pits cause breeze" compactly like natural language. Requires exhaustive enumeration.

Question 21 (2 marks): Define ontological and epistemological commitments in logics.

Commitments define what exists and what is knowable in a logic.

- **Ontological:** Assumes world of independent facts, e.g., propositions as true/false atoms. Commits to rigid structures without relations.
- **Epistemological:** Assumes knowable facts via sentences, but not uncertainties. Limits to certain knowledge, ignoring probabilities.

8. First-Order Logic: Syntax, Semantics, and Knowledge Engineering

Question 22 (10 marks): Detail the syntax of first-order logic (FOL), including constants, functions, predicates, variables, connectives, equality, and quantifiers. Provide examples of atomic sentences, terms, and quantified sentences. Contrast with propositional logic using Wumpus rules (e.g., $\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r,s) \wedge \text{Pit}(r)$).

FOL extends PL with objects/relations for compact, general representations.

- **Constants:** Fixed objects, e.g., $[1,1]$ as square location. Denote specific entities without variation.
- **Functions:** Map to objects, e.g., $\text{RightOf}([1,1]) = [2,1]$. Compute new terms from inputs dynamically.
- **Predicates:** Relations, e.g., $\text{Pit}([2,2])$, $\text{Breeze}([1,1])$. Express properties or connections between objects.
- **Variables:** Placeholders, e.g., x for any square. Allow generalization over domains.
- **Connectives:** Same as PL: $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$. Combine complex expressions hierarchically.
- **Equality (=):** $A = B$ for identity, e.g., $\text{AgentPos}(t) = [1,1]$. Enables exact matching in terms.
- **Quantifiers:** $\forall$ (all), $\exists$ (some), e.g., $\forall x \text{ Pit}(x) \Rightarrow \text{Dangerous}(x)$. Scope over variables for universality.
- **Terms:** Constants/functions/variables, e.g., $\text{Adjacent}([1,1], y)$. Building blocks for predicates.
- **Atomic Sentences:** $\text{Predicate}(\text{term}_1, \dots, \text{term}_n)$, e.g., $\text{At}(\text{Agent}, [1,1], t)$. Basic assertions with structure.
- **Wumpus Contrast:** FOL: $\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r,s) \wedge \text{Pit}(r)$ (one rule); PL needs 16+ per breeze. FOL compactly generalizes patterns.

Question 23 (5 marks): Explain the steps of knowledge engineering in FOL, using the electronic circuits domain as an example. How does it handle queries like circuit functionality?

Systematic process to encode domain in FOL for inference.

- **Task Identification:** Define problem, e.g., verify adder circuit outputs. Clarify goals like sum computation.
- **Knowledge Assembly:** Gather facts like $\text{Connected}(A1, \text{XOR1})$. Collect domain axioms and observations.
- **Vocabulary Choice:** Predicates: $\text{Connected}(x,y)$, $\text{Type}(\text{XOR}, g1)$; functions: $\text{Out}(g)$. Select terms fitting the ontology.
- **Sentences Encoding:** $\forall x \text{Type}(x, \text{XOR}) \Rightarrow \text{Out}(x) = \text{In1}(x) \text{ XOR } \text{In2}(x)$. Formalize rules for components.
- **Query Handling:** Ask $\exists$ values s.t. $\text{KB} \models \text{Sum} = \text{Out}(\text{Adder})$, inferring functionality. Uses inference to prove properties.

Question 24 (2 marks): What are the two types of quantifiers in FOL?

Operators for generality in sentences.

- **Universal ($\forall$):** For all objects, e.g., $\forall x (\text{King}(x) \Rightarrow \text{Mortal}(x))$. Asserts properties hold everywhere.
- **Existential ($\exists$):** There exists, e.g., $\exists x (\text{Gold}(x) \wedge \text{At}(\text{Agent}, x))$. Claims at least one instance.

9. Inference in First-Order Logic

Question 25 (5 marks): Describe universal instantiation (UI) and existential instantiation (EI) rules in FOL inference. Explain how they enable reduction to propositional inference, with an example from Wumpus World (e.g., instantiating $\forall r \text{Pit}(r) \Rightarrow \dots$).

Substitution rules for quantifiers, grounding FOL to PL.

- **UI Rule:** From $\forall x \alpha(x)$, derive $\alpha(c)$ for constant c (no variable capture). Instantiates generals to specifics safely.
- **EI Rule:** From $\exists x \alpha(x)$, introduce Skolem constant c for $\alpha(c)$, unique per step. Handles existence without overlap.
- **Propositional Reduction:** Instantiate all quantifiers to generate ground sentences. Converts FOL to finite PL for algorithms.
- **Efficiency Note:** Exponential in constants, but enables PL algorithms. Practical for bounded domains like Wumpus.
- **Wumpus Example:** $\forall r,s \text{Adjacent}(r,s) \wedge \text{Pit}(r) \Rightarrow \text{Breezy}(s)$; UI $s=[2,1]$, $r=[2,2]$: $\text{Pit}([2,2]) \Rightarrow \text{Breezy}([2,1])$. Applies rule to percepts for local inference.

Question 26 (5 marks): Explain the role of Unification in First-Order Logic (FOL) inference, detailing the concept of the Most General Unifier (MGU).

Unification is fundamental for applying inference rules, such as Resolution or Generalized Modus Ponens, by finding substitutions that make logical expressions identical.

- **Unification Definition:** The $\text{Unify}(p, q)$ algorithm takes two sentences, p and q , and returns a **unifier** σ if one exists. This unifier is a substitution list of variable/term pairs such that when σ is applied, the resulting sentences are identical: $p\sigma = q\sigma$.
- **Most General Unifier (MGU):** For two sentences, there may be multiple possible unifiers. The **Most General Unifier (MGU)** is the substitution that is unique up to the renaming of variables and is considered the single most general one. For instance, given $p = \text{Pit}(x)$ and $q = \text{Pit}(y)$, $\sigma = \{x/y\}$ is the MGU, which is more general than $\sigma = \{x/a, y/a\}$.

Question 27 (5 marks): Define Generalized Modus Ponens (GMP) and state its properties regarding soundness and completeness in FOL KBs.

Generalized Modus Ponens (GMP) is an inference rule used to derive new atomic sentences efficiently in First-Order Logic.

- **Application:** GMP applies to **first-order definite clauses**, which are disjunctions of literals with exactly one positive literal (e.g., or).
- **Soundness:** GMP is **sound**, meaning it only derives sentences that are logically entailed (implies).
- **Completeness:** GMP is **complete** for a Knowledge Base consisting solely of **definite clauses**. This means it can derive all sentences that are entailed by such a KB, or answer every query whose answers are entailed.

Question 28 (5 marks): Compare First-Order Forward Chaining (FC) and Backward Chaining (BC), detailing the properties and critical efficiency issues of FOL Forward Chaining.

Chaining algorithms in FOL apply Generalized Modus Ponens (GMP) and are adapted to handle variables and quantifiers.

- ❖ **Comparison:**
 - ◦ **Forward Chaining (FC):** FC is **data-driven** (like automatic processing, e.g., object recognition) and uses GMP to add new atomic sentences as information streams in. It may perform much work irrelevant to the final goal.
 - ◦ **Backward Chaining (BC):** BC is **goal-driven** (appropriate for problem-solving, e.g., diagnostic questions) and works backward from a query to construct a proof. The complexity of BC can be significantly less than linear in the size of the KB when the goal is specific.
- ❖ **Properties of FOL FC:** FC is **sound** and **complete** specifically for **first-order definite clauses**.
- ❖ **Semi-Decidability and Datalog:** FC terminates in a finite number of iterations for **Datalog**, which is the subset of definite clauses containing no functions. However, if the query is not entailed by the general KB, the process **may not terminate** (making the system semi-decidable).
- ❖ **Efficiency Issues:**
 - **Pattern Matching:** Matching conjunctive premises against known facts is computationally expensive (NP-hard).
 - **Irrelevant Inferences:** FC often generates facts that are irrelevant to the goal.
 - **Optimization:** Efficiency can be improved via **Incremental Forward Chaining** (only matching rules whose premises contain newly added literals) or advanced techniques like the **Rete algorithm** (which retains and gradually completes partial matches as new facts arrive).

Question 29 (5 marks): Describe the First-Order Resolution rule ("Lifted Resolution") and explain its significance regarding completeness for general knowledge bases.

First-Order Resolution (often called Lifted Resolution) is a generalized version of the propositional inference rule that incorporates Unification to handle variables.

- **The Resolution Rule Structure:** The full lifted first-order version of the resolution rule resolves two clauses, and , by finding a unifier such that and the negation of unify: .
 - The resulting resolvent clause is the disjunction of the remaining literals from both original clauses, with the unifier applied: .
- **Prerequisites:** Before applying the rule, the two clauses must be **standardized apart** (variables renamed) so they do not share any variables. Factoring (removal of redundant literals if they are unifiable) is often combined with binary resolution.

- **Role and Significance:** Resolution proves entailment ($\models$) by proving that the negation of the query combined with the KB ($\neg Q \cup KB$) is **unsatisfiable**.
- **Completeness:** Resolution-based inference is **refutation-complete for general KBs**. This means that if a sentence is entailed, resolution can always derive the empty clause from $\neg Q \cup KB$. It can be used to confirm or refute a sentence, making it highly powerful for automated theorem proving.